430.211 Programming Methodology (프로그래밍 방법론)

Taesup Moon

Department of Electrical and Computer Engineering
Seoul National University

Outline

- Chapter 5: Arrays

Ch 5. Arrays

- Array definition:
  - A collection of data of a same type
  - Ex) storing 100 test scores
- First "aggregate" data type we study
  - Means "grouping"
  - `int`, `float`, `double`, `char` are simple data types
- Used for lists of similar items
  - Test scores, temperatures, names, etc.
  - Avoids declaring multiple simple variables
  - Can manipulate "list" as one entity

Declaring Arrays

- Declare the array → allocates memory `int score[5];`
  - Declares array of 5 integers named "score"
  - Similar to declaring five variables:
    `int score[0], score[1], score[2], score[3], score[4]`
  - Array size **must** be a constant
- Individual parts are called as :
  - **Indexed** or **subscripted** variables
  - "**Elements**" of the array
  - Value in brackets called index or subscript
    - Numbered from **0 to (size – 1)**

Accessing Arrays

- Access using index/subscript
  - `cout << score[3];`
- Note two uses of brackets:
  - In declaration, specifies SIZE of array
  - Anywhere else, specifies a subscript
- Subscript need not be literal
  - `int score[MAX_SCORES];`
  - `score[n+1]=99;`
    - If `n` is 2, identical to `score[3]=99`

Array Usage

- Powerful storage mechanism
- Can do commands like:
  - "Do this to i^{th} indexed variable" where i is computed by a program
  - "Display all elements of array score"
  - "Fill elements of array score from user input"
  - "Find highest value in array score"
  - "Find lowest value in array score"

Array Program Example:
Display 5.1 Program Using an Array (1 of 2)

Display 5.1 Program Using an Array

```
1  //Reads in five scores and shows how much each
2  //score differs from the highest score.
3  #include <iostream>
4  using namespace std;

5  int main( )
6  {
7      int i, score[5], max;

8      cout << "Enter 5 scores:\n";
9      cin >> score[0];
10     max = score[0];
11     for (i = 1; i < 5; i++)
12     {
13         cin >> score[i];
14         if (score[i] > max)
15             max = score[i];
16         //max is the largest of the values score[0],..., score[i].
17     }
```

Array Program Example:
Display 5.1 Program Using an Array (2 of 2)

```
18      cout << "The highest score is " << max << endl
19           << "The scores and their\n"
20           << "differences from the highest are:\n";
21      for (i = 0; i < 5; i++)
22          cout << score[i] << " off by "
23               << (max - score[i]) << endl;
24      return 0;
25  }
```

For loop is ideally suited to cout through an array!

Sample Dialogue

```
Enter 5 scores:
5 9 2 10 6
The highest score is 10
The scores and their
differences from the highest are:
5 off by 5
9 off by 1
2 off by 8
10 off by 0
6 off by 4
```

Major Array Pitfall

- Array indexes always start with zero!
- Zero is the "first" number to computer scientists
- C++ will "let" you go beyond range
  - Unpredictable results
  - Compiler will not detect these errors!
- Up to programmer to "stay in range"

Major Array Pitfall Example

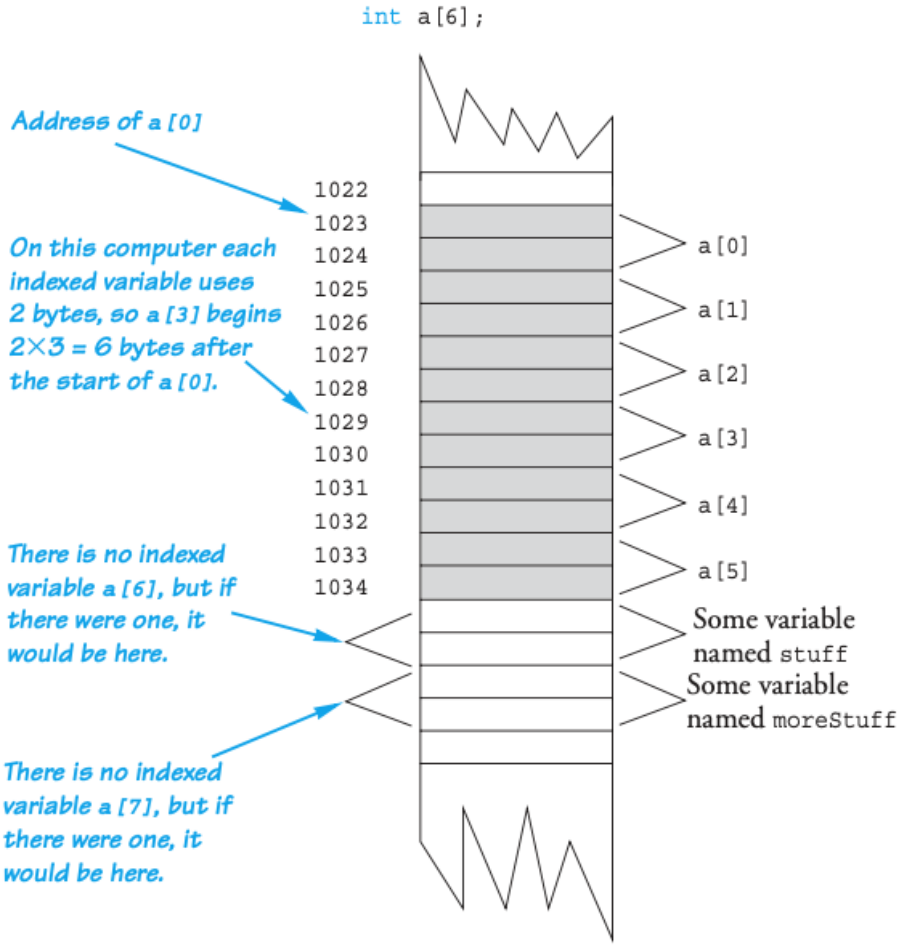
- Indexes range from 0 to (array_size – 1)
  - Example:
    ```
    int a[6]; // 6 is array size
    ```
    // Declares array of 6 `int` values called temperature
    - They are indexed as: `a[0], a[1], … ,a[5]`
  - Common mistake: `a[6] = 5;`
    - Index 6 is "out of range"!
    - No warning, possibly disastrous results (could have changed some other variable's value)

Arrays in Memory

- Recall simple variables:
  - Allocated memory in an "address"
- Array declarations allocate memory for entire array
- Sequentially-allocated
  - Means addresses allocated "back-to-back"
  - Allows indexing calculations
    - Simple "addition" from the array beginning (index 0)

An Array in Memory

Display 5.2 An Array in Memory



Defined Constant as Array Size

- Can use defined/named constant for array size
- Example:

```
const int NUMBER_OF_STUDENTS = 5;
int score[NUMBER_OF_STUDENTS];
```

  - Improves readability, versatility, and maintainability
- What if we don't know the total number of students?

```
cout << "Enter number of students:\n";
cin >> number;
int score[number]; //ILLEGAL ON MANY COMPILERS!
```

  - Try not to do this way (use MAX upperbound)
  - In Ch.10, we will cover different kind of array whose size can be determined when the program is run (pointers)

Initializing Arrays

- As simple variables can be initialized at declaration:

```
int price = 0; // 0 is initial value
```

- Arrays can as well: `int children[3] = {2, 12, 1};`
  - Equivalent to the following:

```
int children[3];
children[0] = 2;
children[1] = 12;
children[2] = 1;
```

Auto-Initializing Arrays

- If fewer values than array size are supplied:
  - Fills from the beginning
  - Fills the "rest" with **zero** of array base type
- If array-size is left out
  - Declares array with size required based on the number of initialization values
  - Example:

```
int b[] = {5, 12, 11};
```

    - Allocates array b to size 3
    - What's the difference with

```
int b[4] = {5, 12, 11};?
```

Arrays in Functions

- As arguments to functions
  - Indexed variables
    - An individual "element" of an array can be a function parameter
  - Entire arrays
    - All array elements can be passed as "one entity"
- As a return value of a function
  - Can be done (with a pointer) → Chapter 10

Indexed Variables as Arguments

- Indexed variable is handled the same as a simple variable of array base type
- Ex) given this function declaration:

```
void myFunction(double par1);
```

- And these declarations:

```
int i;  double n, a[10];
```

- Can make these function calls:

```
myFunction(i); // i is converted to double
myFunction(a[3]); // a[3] is double
myFunction(n); // n is double
```

Entire Arrays as Arguments

- Formal parameter can be an entire array
  - In this case, the argument passed in a function call is the array name
  - Called "array parameter"
- Send size of array as well
  - Typically done as the second parameter
  - Simple `int` type formal parameter

Entire Array as Argument Example: **Display 5.3** Function with an Array Parameter

Display 5.3 Function with an Array Parameter

Function Declaration

```
void fillUp(int a[], int size);
//Precondition: size is the declared size of the array a.
//The user will type in size integers.
//Postcondition: The array a is filled with size integers
//from the keyboard.
```

Formal array parameter
→ Not a call-by-reference parameter, but similar in some sense

Function Definition

```
void fillUp(int a[], int size)
{
    cout << "Enter " << size << " numbers:\n";
    for (int i = 0; i < size; i++)
        cin >> a[i];
    cout << "The last array index used is " << (size - 1) << endl;
}
```

Entire Array as Argument Example

- Continuing from the previous example
  - Suppose in the `main()` function definition, consider the call

No bracket!

```
int score[5], numberOfScores = 5;
fillUp(score, numberOfScores);
```

  - 1st argument is the **entire array**
  - 2nd argument is indicating the **size of the array**
  - The function call is equivalent to

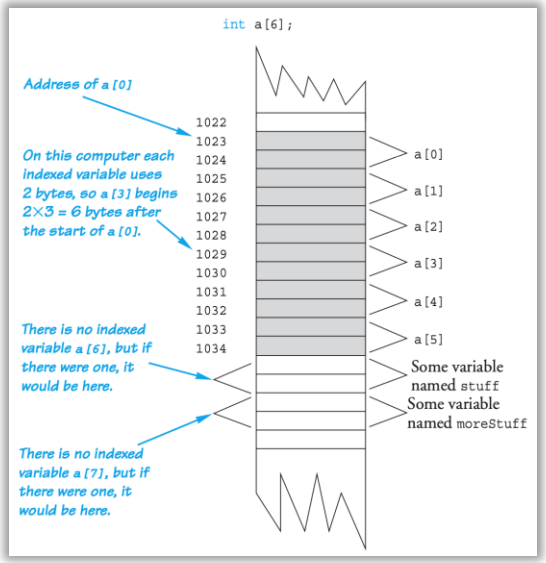
```
{
    size = 5;
    cout << "Enter " << size << " numbers:\n";
    for (int i = 0; i < size; i++)
        cin >> score[i];
    cout << "The last array index used is " << (size - 1) << endl;
}
```

5 is the value of **numberOfScores**

Formal parameter `a` is replaced with `score`!
→ Similar to call-by-reference!

Array as Argument: How?

- Again, recall the memory allocation for array
  - When `score[5]` is declared, the computer remembers only the address of `score[0]`
  - The address of the rest can be deduced
- An array has three parts
  1. The address of the first indexed variable
  2. The base type of the array
  3. The size of the array



- When an array argument is used,
  - Only **the first** part is passed (base type matched!)
  - The size of the array is NOT passed

Array as Argument: How?

- Thus, **the size of the array** must be passed!
  - Array parameter is a weak form of call-by-reference
  - Everything of array is told except for the size
- Why such strange definition? (no bracket+size)
  - Can **re-use** the same function for **any** size of array

```
int score[5], time[10];
```

```
fillUp(score, 5);
fillUp(time, 10);
```

- Sidenote
  - The array argument is passing the pointer to its first (zeroth) index variable (more in Ch. 10)

The `const` Parameter Modifier

- Array parameter passes the address of 1st element
  - Similar to call-by-reference
- Function can then modify array!
  - Often desirable, sometimes not!
- To protect array contents from modification
  - Use "`const`" modifier before array parameter
    - Called "constant array parameter"
    - Tells compiler to "not allow" modifications

The `const` Parameter Modifier : Example

```
void showTheWorld(int a[], int sizeOfa)
//Precondition: sizeOfa is the declared size of the array a.
//All indexed variables of a have been given values.
//Postcondition: The values in a have been written to the screen.
{
    cout << "The array contains the following values:\n";
    for (int i = 0; i < sizeOfa; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

```
void showTheWorld(const int a[], int sizeOfa)
//Precondition: sizeOfa is the declared size of the array a.
//All indexed variables of a have been given values.
//Postcondition: The values in a have been written to the screen.
{
    cout << "The array contains the following values:\n";
    for (int i = 0; i < sizeOfa; a[i]++)
        cout << a[i] << " ";
    cout << endl;
}
```

Mistake, but the compiler will not catch it unless you use the `const` modifier.

If `const` is not used, the function will result in an infinite loop!
→ safety check for preventing unwanted errors

Functions that Return an Array

- Functions cannot return arrays the same way as simple types are returned
- Requires use of a "pointer"
- Will study in Chap.10.

Multidimensional Arrays

- Arrays with more than one index

```
char page[30][100];
```

  - Two indexes: An "array of arrays"
  - Visualize as:

```
page[0][0], page[0][1], ..., page[0][99]
page[1][0], page[1][1], ..., page[1][99]
page[2][0], page[2][1], ..., page[2][99]
          .
          .
          .
page[29][0], page[29][1], ..., page[29][99]
```

  - A one-dimensional array of size 30, whose base type is a one-dimensional array of `char`s of size 100!

- C++ allows any finite number of indexes

Multidimensional Array Function Parameters

- Similar to one-dimensional array
  - 1st dimension size is not given
    - Provided as second parameter
  - 2nd dimension (3rd, 4th, …) sizes ARE given as constant
    - Part of description of the base type!
- Example:

```
void displayPage(const char p[][100], int sizeDimension1)
{
    for (int index1 = 0; index1 < sizeDimension1; index1++)
    { //Printing one line:
        for (int index2 = 0; index2 < 100; index2++)
            cout << p[index1][index2];
        cout << endl;
    }
}
```

  - Then, call `displayPage(page,30);`

Summary I

- Array is collection of "same type" data
- Indexed variables of array used just like any other simple variables
- for-loop "natural" way to traverse arrays
- Programmer responsible for staying "in bounds" of array
- Array parameter is a "new" kind
  - Similar to call-by-reference

Summary 2

- Array elements stored sequentially
  - "Contiguous" portion of memory
  - Only address of 1st element is passed to functions
- Constant array parameters
  - Prevent modification of array contents
- Multidimensional arrays
  - Create "array of arrays"

Thank you!

Web: http://mindlab-snu.github.io
E-mail: tsmoon@snu.ac.kr